

PROYECTO FINAL DE INTELIGENCIA ARTIFICIAL

Resolución del TSP mediante Algoritmos Genéticos

Informe profesional mejorado y sustentado por literales

Grupo 5

Nancy Altamirano | Gustavo Berru | Raquel Pacheco | Kevin Viteri

Carrera: Ciencia de Datos

Mayo de 2026

Tema seleccionado: resolución del Problema del Viajante de Comercio (TSP) mediante un Algoritmo Genético Generacional Elitista con representación por permutaciones, crossover OX, mutación por inversión y refinamiento final 2-opt.

Índice del informe

Resumen ejecutivo

Objetivo, contexto y configuración del experimento

a) Representación de individuos como permutaciones válidas

b) Operadores genéticos seleccionados y justificación

c) Análisis de la evolución del fitness

d) Medición y análisis de diversidad poblacional

e) Evaluación de la calidad de las soluciones

f) Análisis experimental de parámetros

Conclusiones técnicas

Resumen de participación del Grupo 5

Referencias de apoyo

Resumen ejecutivo

Este informe presenta la solución del Problema del Viajante de Comercio (TSP) mediante un Algoritmo Genético Generacional Elitista. El TSP requiere construir una ruta cerrada que visite cada ciudad exactamente una vez y retorne al punto inicial minimizando la distancia total. Debido a que el número de rutas posibles crece factorialmente con el número de ciudades, el uso de búsqueda exhaustiva resulta ineficiente para instancias medianas; por ello se aplica una técnica evolutiva capaz de explorar múltiples rutas candidatas y mejorar progresivamente la calidad de las soluciones.

Se trabajaron los 10 datasets entregados, con tamaños entre 80 y 88 ciudades. No se agregaron ciudades artificiales, porque el objetivo fue respetar los datos reales suministrados. El algoritmo principal usó población de 100 individuos, 250 generaciones, selección por torneo, crossover OX, mutación por inversión, elitismo de dos individuos y refinamiento local final 2-opt.

El resultado fue válido en todos los datasets: todas las rutas finales fueron permutaciones completas, sin ciudades repetidas ni faltantes. En comparación con la heurística Nearest Neighbor, la distancia promedio pasó de 1,638.67 a 1,509.45, lo que representa una mejora relativa promedio de -7.83% respecto a dicha heurística. Como el óptimo global certificado no fue proporcionado para los datasets, el error relativo se reporta contra la heurística de referencia, tal como permite el literal e).

Idea clave para la presentación:

El AG elitista fue elegido como estructura principal porque conserva las mejores rutas encontradas y evita retrocesos por azar; OX no es un algoritmo alternativo al AG elitista, sino el operador de crossover usado dentro de ese algoritmo para preservar rutas válidas en forma de permutaciones.

Objetivo, contexto y configuración del experimento

El objetivo del proyecto es resolver el TSP con un enfoque de Inteligencia Artificial basado en Algoritmos Genéticos y demostrar, de forma explícita, que se cumplen los literales a) hasta f) solicitados en el enunciado. Por ello, el informe está organizado en el mismo orden de los literales y cada sección incluye qué se hizo, por qué se hizo, cómo se verificó y qué evidencia se obtuvo.

Configuración base utilizada en la ejecución principal:

Tabla 1. Configuración base del algoritmo.

Parámetro	Valor usado	Justificación
Representación	Permutación de ciudades	Cada cromosoma contiene todos los índices de ciudad una sola vez.
Población	100 individuos	Permite diversidad inicial suficiente sin elevar demasiado el costo computacional.
Generaciones	250	Da tiempo para observar convergencia, estancamiento y evolución del fitness.
Selección	Torneo k = 3	Mantiene presión selectiva moderada sin eliminar completamente la diversidad.
Crossover	OX, probabilidad 0.90	Conserva orden relativo y evita rutas inválidas en problemas de permutación.
Mutación	Inversión, probabilidad 0.15	Modifica segmentos de la ruta sin repetir ni eliminar ciudades.
Elitismo	2 mejores individuos	Protege las mejores rutas entre generaciones.
Mejora local	2-opt final	Refina la mejor ruta eliminando cruces o enlaces innecesarios.

Datasets trabajados

Tabla 2. Datasets procesados.

Dataset	Cantidad de ciudades
cities_100_101112	85

cities_100_123	82
cities_100_131415	88
cities_100_161718	80
cities_100_192021	80
cities_100_222324	82
cities_100_252627	84
cities_100_282930	83
cities_100_456	84
cities_100_789	80

Aunque los archivos tienen menos de 100 ciudades en algunos casos, se conservaron exactamente como fueron entregados. Esto evita alterar la dificultad real de cada instancia y permite que los resultados sean reproducibles con los CSV originales.

a) Representación de cada individuo como una permutación válida de ciudades

Qué pide el literal:

- Representar cada individuo como una permutación válida de ciudades.
- Garantizar que las soluciones generadas en la inicialización, el crossover y la mutación sean válidas.
- Demostrar explícitamente que las rutas no tienen ciudades repetidas.
- Demostrar explícitamente que las rutas no tienen ciudades faltantes.

Respuesta implementada:

Cada individuo se representó como un vector ordenado de índices de ciudades. Si un dataset tiene n ciudades, un cromosoma válido contiene exactamente los valores $0, 1, 2, \dots, n-1$. La posición del índice dentro del vector define el orden de visita; por tanto, el cromosoma no almacena coordenadas, sino la secuencia de ciudades que construye la ruta.

Ejemplo conceptual: $[4, 1, 7, 0, 3, 2, 6, 5]$ significa que la ruta inicia en la ciudad 4, luego visita la ciudad 1, después la ciudad 7, y así sucesivamente hasta volver a la ciudad inicial. Esta representación es correcta para el TSP porque el problema no consiste en decidir si una ciudad se visita o no, sino en decidir en qué orden se visitan todas las ciudades.

Garantía de validez por etapa:

Tabla 3. Control de validez en inicialización, crossover y mutación.

Etapas	Mecanismo aplicado	Cómo garantiza validez
Inicialización	Se genera cada cromosoma con una permutación aleatoria de 0 a $n-1$. También se incluye una ruta semilla de Nearest Neighbor para iniciar con una solución razonable.	No puede haber repetidas ni faltantes porque la función de permutación genera cada índice una sola vez.
Crossover	Se aplica Order Crossover (OX). Copia un segmento de un padre y completa los espacios con ciudades del segundo padre en el orden en que aparecen.	OX evita duplicados porque no inserta genes ya usados y completa con los faltantes.
Mutación	Se aplica mutación por inversión. Selecciona dos posiciones e invierte el segmento entre ellas.	La inversión solo cambia el orden; no crea ni elimina ciudades.
Validación defensiva	El código verifica cada ruta con <code>is_valid_permutation</code> y tiene una reparación defensiva si apareciera una ruta inválida.	En la ejecución real no se activó reparación porque OX + inversión conservaron la validez.

Tabla 4. Evidencia de rutas válidas por dataset.

Dataset	n	Sin repetidas	Sin faltantes	Hijos inválidos	Reparados	Poblaciones inválidas
cities_100_101112	85	Sí	Sí	0	0	0
cities_100_123	82	Sí	Sí	0	0	0
cities_100_131415	88	Sí	Sí	0	0	0
cities_100_161718	80	Sí	Sí	0	0	0
cities_100_192021	80	Sí	Sí	0	0	0

cities_100_222324	82	Sí	Sí	0	0	0
cities_100_252627	84	Sí	Sí	0	0	0
cities_100_282930	83	Sí	Sí	0	0	0
cities_100_456	84	Sí	Sí	0	0	0
cities_100_789	80	Sí	Sí	0	0	0

Sustentación del literal a): la validez no se dejó al azar. La estructura de permutación, el crossover OX y la mutación por inversión fueron seleccionados porque respetan la naturaleza combinatoria del TSP. La tabla anterior confirma que todos los datasets terminaron con rutas válidas; además, se registraron 0 hijos inválidos, 0 reparaciones y 0 generaciones con población inválida. Por tanto, se cumple explícitamente la condición de no repetir ciudades y no omitir ciudades.

b) Definición de operadores genéticos y justificación del uso

Qué pide el literal:

- Definir un operador de crossover válido para permutaciones, como PMX, OX o CX.
- Definir un operador de mutación válido, como swap, inversión o scramble.
- Justificar por qué esos operadores son adecuados para el TSP.

Respuesta implementada:

El algoritmo elegido fue un Algoritmo Genético Generacional Elitista. Dentro de ese algoritmo se utilizaron selección por torneo, crossover OX, mutación por inversión y elitismo. Es importante diferenciar la variante del algoritmo de sus operadores: el elitismo define cómo se conserva la mejor solución entre generaciones; OX define cómo se cruzan dos rutas padres para formar un hijo válido.

Análisis conceptual:

La elección no fue entre OX y el AG elitista, porque pertenecen a niveles diferentes de la solución. El AG generacional elitista es el enfoque principal de búsqueda; OX es el operador de crossover usado dentro de ese AG para recombinar rutas sin romper la estructura de permutación. Por tanto, la formulación correcta es: “Algoritmo Genético Generacional Elitista con crossover OX y mutación por inversión”.

Tabla 5. Operadores genéticos y justificación técnica.

Operador o componente	Funcionamiento	Justificación
Selección por torneo	Se eligen k individuos al azar y se reproduce el de menor distancia.	Controla la presión selectiva y permite que las mejores rutas tengan más probabilidad de reproducirse sin eliminar toda la diversidad.
Crossover OX	Copia un segmento de un padre y completa con el orden relativo del otro padre.	Es válido para permutaciones: evita ciudades repetidas y faltantes, y conserva información de orden, que es crítica en el TSP.
Mutación por inversión	Invierte un segmento de la ruta.	Mantiene todos los genes y solo cambia el orden de visita; además, es coherente con mejoras típicas del TSP.
Elitismo	Copia los mejores individuos directamente a la siguiente generación.	Evita que una buena ruta se pierda por azar durante crossover o mutación.
2-opt final	Invierte segmentos si reduce la distancia total.	Refina la mejor ruta hallada por el AG; por eso el enfoque puede considerarse híbrido o memético en la etapa final.

Sustentación del literal b): OX fue seleccionado porque el TSP es sensible al orden de visita. Un cruce tradicional podría producir cromosomas inválidos, por ejemplo con ciudades repetidas y otras ausentes. OX reduce ese riesgo porque preserva un segmento de una ruta padre y completa la solución con las ciudades que faltan siguiendo el orden del otro padre. La mutación por inversión se seleccionó porque explora cambios locales sin destruir la estructura de permutación. El elitismo se incorporó para proteger la mejor solución alcanzada y estabilizar la convergencia.

c) Análisis de la evolución del fitness

Qué pide el literal:

- Graficar por generación el mejor fitness, el fitness promedio y el peor fitness.
- Analizar la convergencia del algoritmo.
- Analizar la presencia de estancamiento o convergencia prematura.

Definición de fitness:

El objetivo del TSP es minimizar la distancia total. Para adaptar el problema al lenguaje de fitness, se definió $\text{fitness} = 1 / \text{distancia_total}$. Con esta definición, una ruta más corta tiene mayor fitness. En cada generación se registraron tres métricas: mejor fitness, fitness promedio y peor fitness. Estas tres curvas permiten observar si la población mejora, si se concentra alrededor de buenas soluciones o si todavía mantiene individuos de baja calidad.

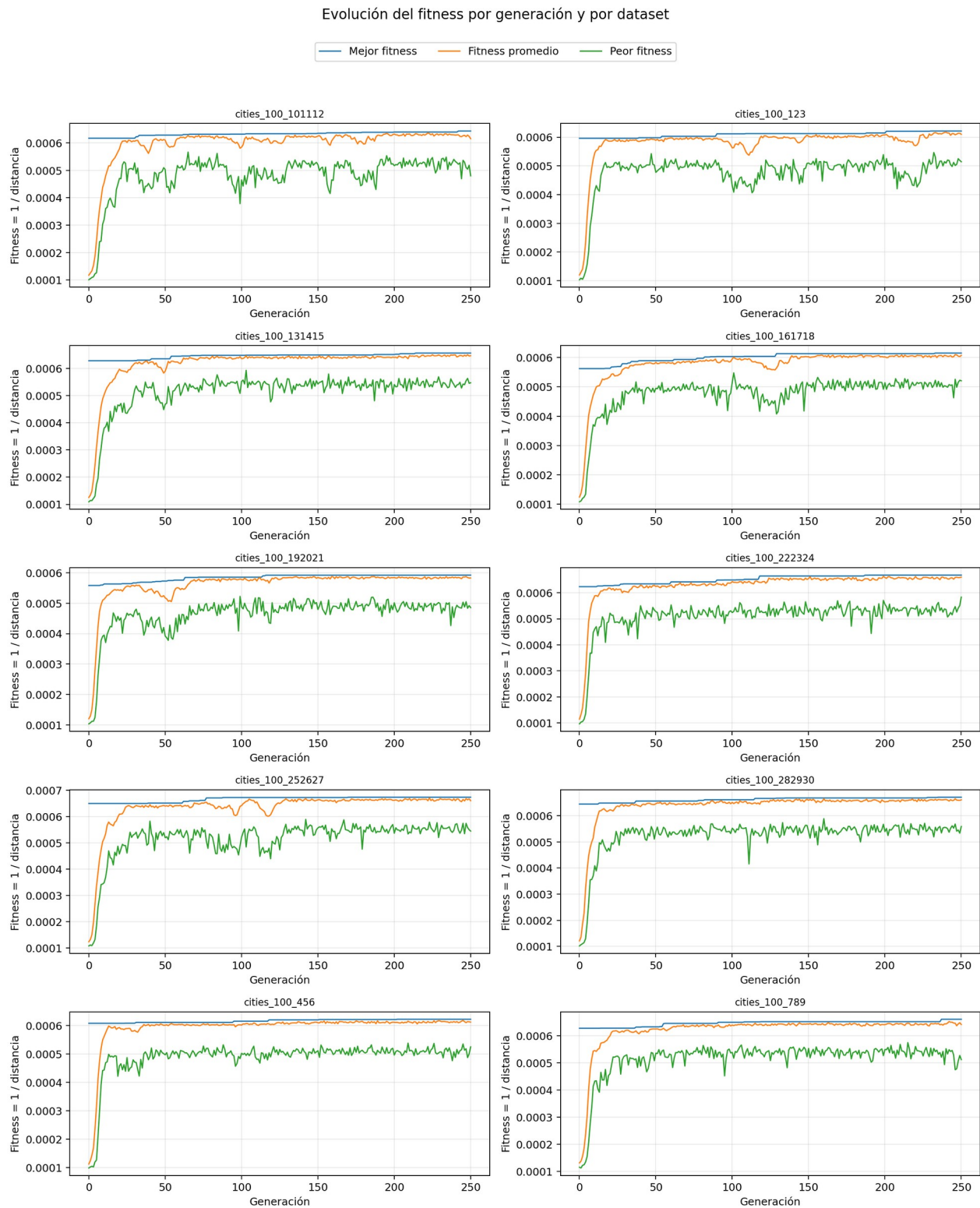


Figura 1. Evolución por generación del mejor fitness, fitness promedio y peor fitness para los 10 datasets.

Análisis de la gráfica de fitness:

En los 10 datasets se observa el comportamiento esperado de un algoritmo genético: el mejor fitness crece de manera rápida al inicio, porque la población encuentra rutas claramente mejores que las aleatorias; después, las mejoras se vuelven más pequeñas y la curva se estabiliza. El fitness promedio tiende a acercarse al mejor fitness, lo que evidencia convergencia

de la población hacia rutas de mayor calidad. El peor fitness también mejora o se estabiliza porque la selección y el reemplazo generacional reducen la presencia de rutas de mala calidad.

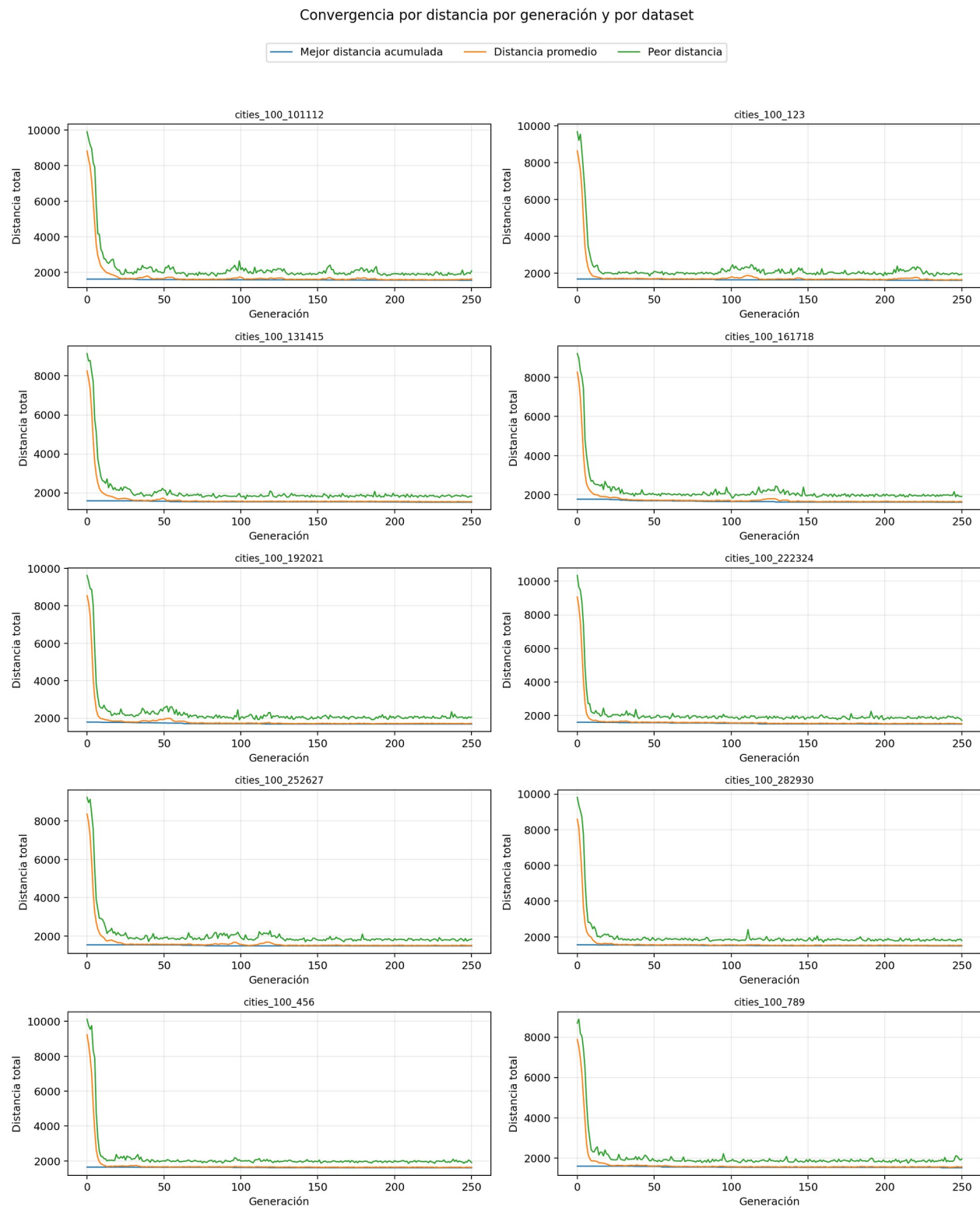


Figura 2. Convergencia por distancia: mejor distancia acumulada, distancia promedio y peor distancia por generación.

Tabla 6. Evidencia de convergencia y estancamiento por dataset.

Dataset	Mejor generación	Gen. finales sin mejora	Únicos finales (%)	Hamming final (%)	Diagnóstico
cities_100_101112	242	8	35.0	39.0	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.

cities_100_123	230	20	16.0	7.0	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.
cities_100_131415	214	36	19.0	11.0	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.
cities_100_161718	233	17	17.0	6.2	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.
cities_100_192021	225	25	20.0	14.3	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.
cities_100_222324	190	60	16.0	6.7	Posible convergencia prematura: largo periodo sin mejora y diversidad final baja.
cities_100_252627	170	80	17.0	7.9	Posible convergencia prematura: largo periodo sin mejora y diversidad final baja.
cities_100_282930	235	15	15.0	7.7	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.
cities_100_456	202	48	18.0	13.3	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.
cities_100_789	237	13	24.0	24.3	Convergencia activa: se observaron mejoras dentro del tramo final del algoritmo.

Análisis de convergencia y estancamiento:

El criterio de convergencia no se basó únicamente en observar las curvas. También se registró la generación en la que apareció la mejor solución, la cantidad de generaciones finales sin mejora y la diversidad final. En 8 de los 10 datasets se clasificó la convergencia como activa o con estancamiento parcial controlado, porque todavía existieron mejoras o diversidad residual. En 2 datasets se identificó posible convergencia prematura, debido a un tramo final largo sin mejora y diversidad final baja. Esto no invalida el algoritmo; más bien muestra que, en esas instancias, sería conveniente aumentar mutación, población o generaciones si se quisiera seguir explorando.

Sustentación del literal c): se graficaron las tres curvas de fitness solicitadas para cada dataset y se complementaron con curvas de distancia para interpretar la convergencia. Además, el estancamiento se analizó con indicadores cuantitativos y no solo con apreciación visual.

d) Medición y análisis de diversidad poblacional

Qué pide el literal:

- Definir una métrica de diversidad, como distancia promedio entre individuos y/o porcentaje de individuos únicos.
- Graficar la diversidad a lo largo de las generaciones.
- Analizar la relación entre diversidad y calidad de solución.
- Analizar la relación entre diversidad y convergencia.

Métricas de diversidad usadas:

- **Porcentaje de individuos únicos:** mide qué proporción de la población contiene rutas distintas. Si baja demasiado, la población se vuelve homogénea.
- **Distancia Hamming promedio normalizada:** mide el porcentaje promedio de posiciones diferentes entre pares de rutas. Sirve como distancia promedio entre individuos en una representación por permutaciones.

Evolución de la diversidad poblacional por dataset

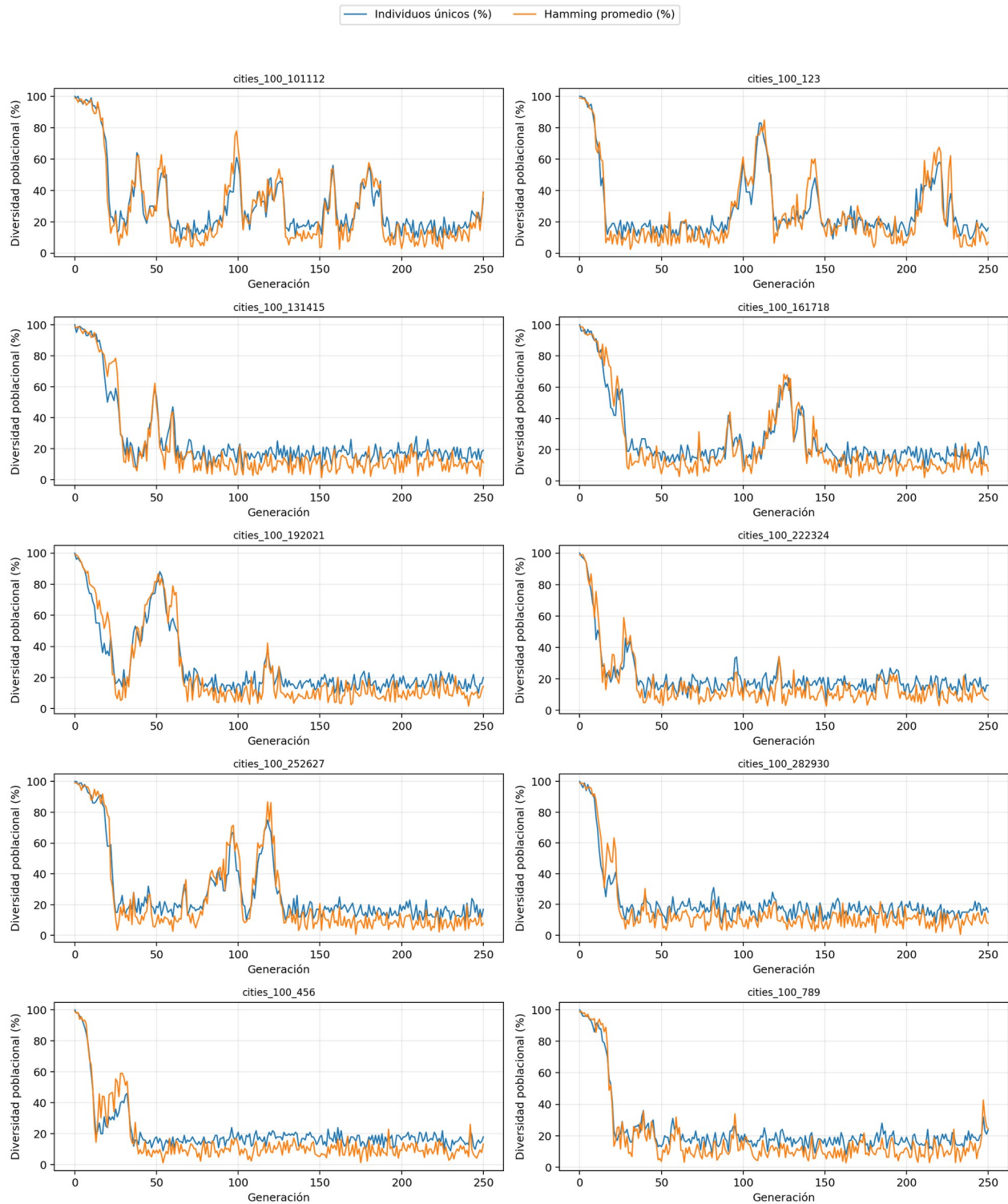


Figura 3. Evolución de diversidad poblacional por generación: individuos únicos y distancia Hamming promedio.

Análisis de la diversidad a lo largo de las generaciones:

La diversidad disminuye conforme avanza el algoritmo porque la selección y el elitismo favorecen la reproducción de rutas de mejor calidad. Esta reducción es normal: el algoritmo empieza explorando muchas rutas distintas y luego explota las regiones más prometedoras. Sin embargo, si la diversidad cae demasiado pronto, puede aparecer convergencia prematura, porque la población deja de explorar rutas alternativas.

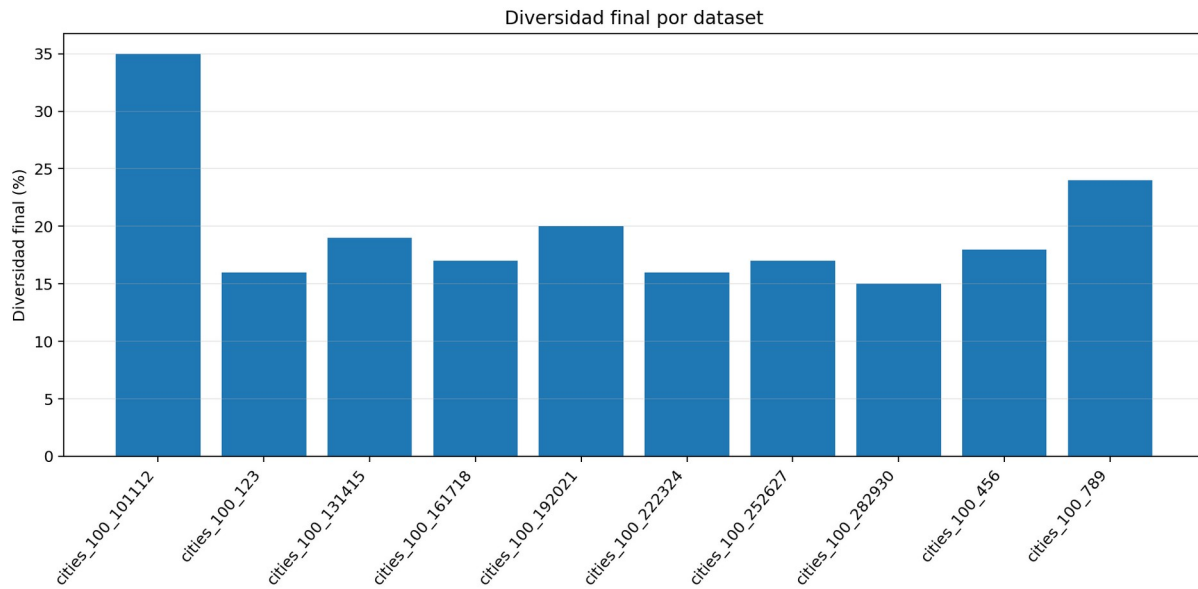


Figura 4. Porcentaje final de individuos únicos por dataset.

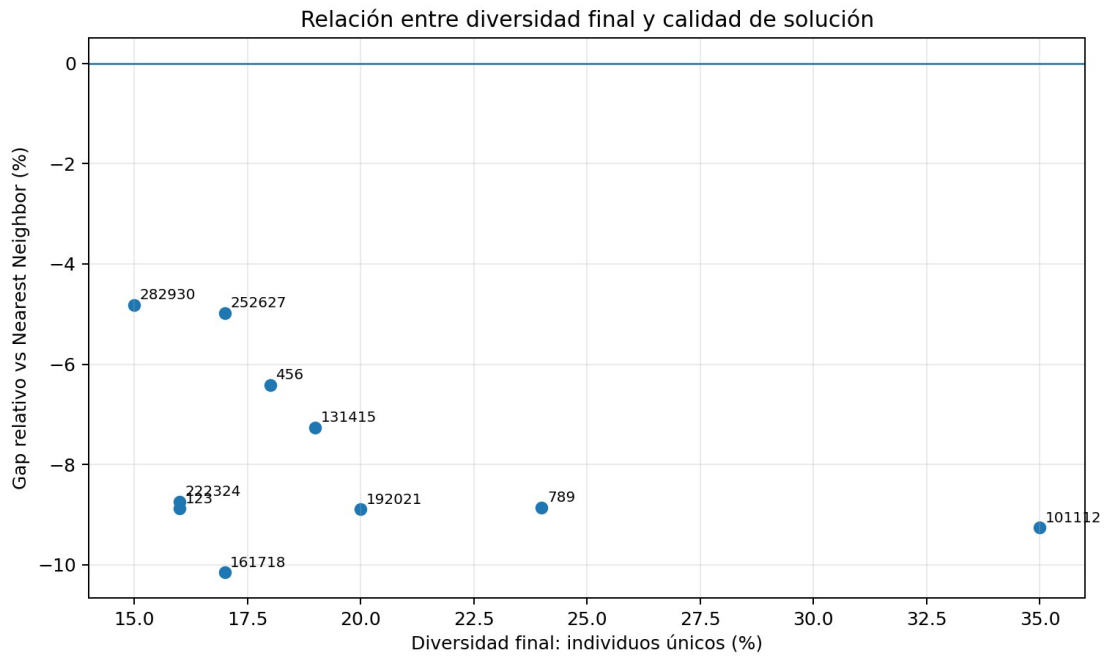


Figura 5. Relación entre diversidad final y calidad de solución medida como gap relativo frente a Nearest Neighbor.

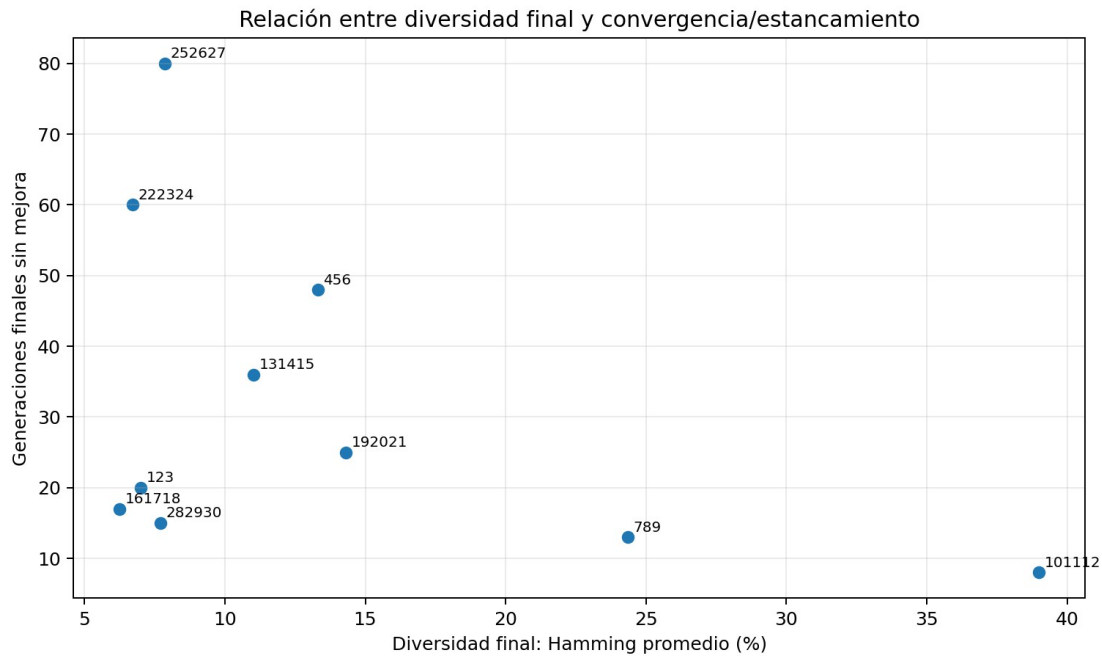


Figura 6. Relación entre diversidad final y convergencia/estancamiento medida por generaciones finales sin mejora.

Tabla 7. Relación cuantitativa entre diversidad, calidad y convergencia.

Dataset	Corr. únicos vs distancia	Corr. Hamming vs distancia	Corr. únicos vs no mejora
cities_100_101112	0.620	0.563	-0.234
cities_100_123	0.087	0.028	-0.260
cities_100_131415	0.723	0.762	-0.153
cities_100_161718	0.695	0.674	-0.268
cities_100_192021	0.758	0.784	-0.426
cities_100_222324	0.491	0.539	-0.300
cities_100_252627	0.428	0.366	-0.367
cities_100_282930	0.604	0.642	-0.228
cities_100_456	0.438	0.530	-0.347
cities_100_789	0.611	0.610	-0.286

Análisis diversidad-calidad:

La relación no debe interpretarse como “más diversidad siempre es mejor”. Al inicio, una diversidad alta es necesaria para explorar rutas diferentes. En etapas finales, una diversidad moderadamente menor puede ser positiva porque indica que la población se concentra en buenas soluciones. El riesgo aparece cuando la diversidad cae demasiado temprano y el algoritmo ya no mejora. En los resultados, los gaps negativos frente a Nearest Neighbor muestran que el algoritmo obtuvo soluciones de buena calidad aun cuando la diversidad final bajó en varios datasets.

Análisis diversidad-convergencia:

Cuando la diversidad final es baja y el número de generaciones sin mejora es alto, existe evidencia de estancamiento. Esto ocurrió principalmente en los datasets señalados como posible convergencia prematura. En términos prácticos, esta información permite defender decisiones de ajuste: aumentar la probabilidad de mutación, ampliar la población o prolongar generaciones puede reintroducir exploración si la diversidad cae demasiado pronto.

Sustentación del literal d): se definieron dos métricas de diversidad, se graficaron a lo largo de las generaciones y se analizó su relación con calidad y convergencia mediante gráficas y correlaciones. Por tanto, el literal no queda solo en una definición, sino que se evalúa con evidencia experimental.

e) Evaluación de la calidad de las soluciones

Qué pide el literal:

- Comparar los resultados del AG con al menos una heurística simple, por ejemplo Nearest Neighbor.
- Reportar fitness final y error relativo cuando sea posible.

Heurística de comparación:

Se utilizó Nearest Neighbor como heurística simple. Esta heurística inicia en una ciudad y, en cada paso, elige la ciudad no visitada más cercana. Para hacer una comparación más justa, se probó cada ciudad como posible inicio y se conservó la mejor ruta obtenida. Además, se calculó una versión Nearest Neighbor + 2-opt para comparar el AG contra una heurística refinada.

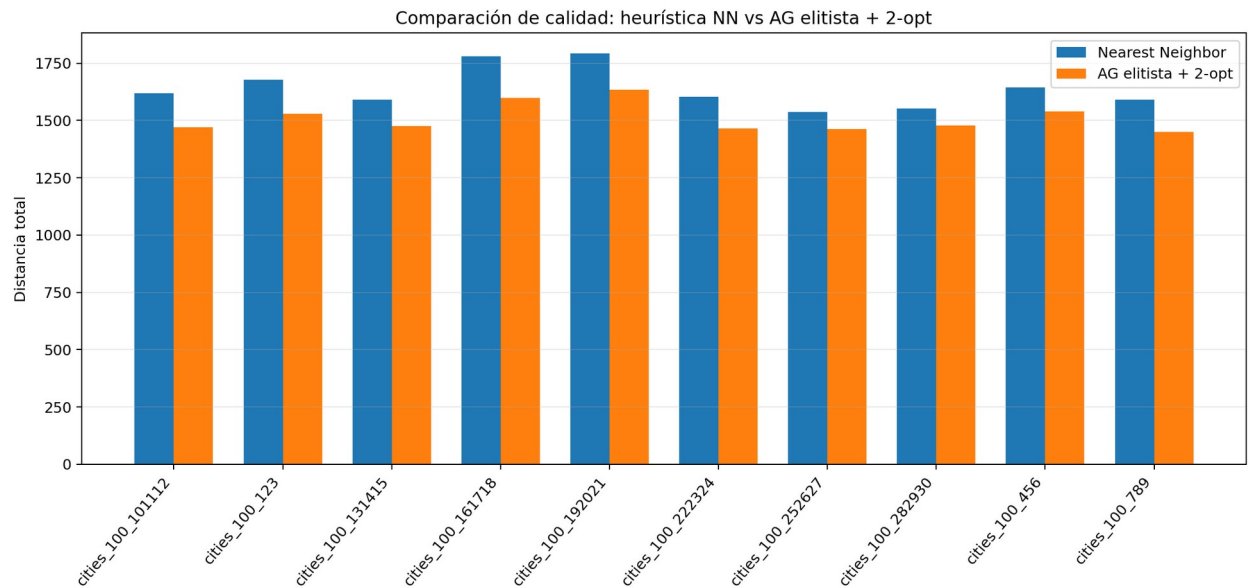


Figura 7. Comparación de distancia final: Nearest Neighbor vs AG elitista + 2-opt.

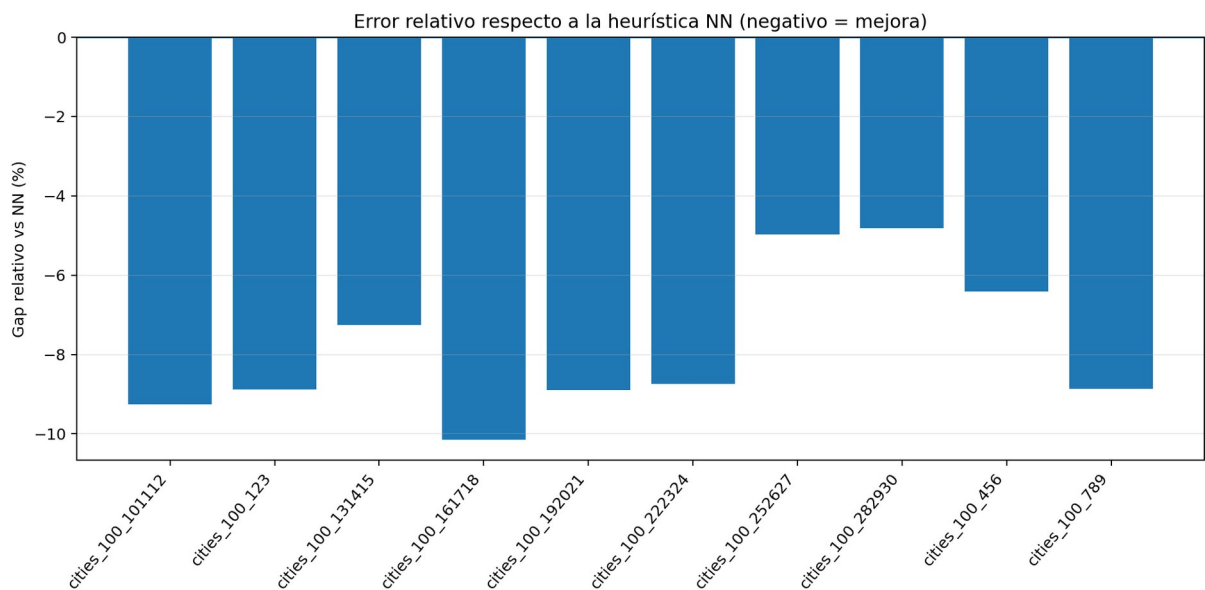


Figura 8. Error relativo frente a Nearest Neighbor. Valores negativos representan mejora del AG.

Tabla 8. Calidad de solución, fitness final y error relativo.

Dataset	n	NN dist.	NN+2opt dist.	AG+2opt dist.	Fitness final	Error vs NN	Error vs NN+2opt	Ruta válida
cities_100_101112	85	1,619.41	1,487.02	1,469.42	0.000681	-9.26	-1.18	Sí
cities_100_123	82	1,676.68	1,536.72	1,527.83	0.000655	-8.88	-0.58	Sí
cities_100_131415	88	1,590.97	1,472.80	1,475.42	0.000678	-7.26	0.18	Sí
cities_100_161718	80	1,778.42	1,586.98	1,597.80	0.000626	-10.16	0.68	Sí
cities_100_192021	80	1,792.02	1,564.55	1,632.69	0.000612	-8.89	4.36	Sí
cities_100_222324	82	1,604.02	1,493.68	1,463.85	0.000683	-8.74	-2.00	Sí
cities_100_252627	84	1,537.77	1,491.85	1,461.27	0.000684	-4.97	-2.05	Sí
cities_100_282930	83	1,551.43	1,494.41	1,476.72	0.000677	-4.82	-1.18	Sí
cities_100_456	84	1,644.44	1,544.33	1,538.99	0.000650	-6.41	-0.35	Sí
cities_100_789	80	1,591.55	1,477.00	1,450.54	0.000689	-8.86	-1.79	Sí

Análisis de calidad:

El AG elitista + 2-opt superó a Nearest Neighbor en todos los datasets, porque los errores relativos frente a NN fueron negativos. En promedio, la distancia se redujo en -7.83% respecto a NN. Frente a NN+2-opt, el promedio fue -0.39%, lo que indica que el AG también fue competitivo frente a una heurística simple refinada, aunque en algunos datasets NN+2-opt quedó ligeramente por debajo. Esto es esperable porque 2-opt es una mejora local fuerte y el AG no garantiza óptimo global, sino buenas soluciones aproximadas.

Nota sobre el error relativo:

Como los datasets no incluyen una solución óptima certificada, no sería correcto reportar error relativo contra el óptimo global. Por transparencia, el error relativo se reportó respecto a la heurística Nearest Neighbor y adicionalmente respecto a Nearest Neighbor + 2-opt. Esta decisión cumple el literal e) "cuando sea posible" y evita afirmar una precisión que no fue verificada.

Sustentación del literal e): se comparó el AG contra una heurística simple, se reportó el fitness final y se calculó error relativo contra la referencia disponible. Las rutas finales fueron válidas en todos los casos, por lo que la comparación no mezcla soluciones inválidas ni incompletas.

f) Análisis experimental de parámetros

Qué pide el literal:

- Reportar resultados en tablas.
- Analizar el tamaño de población.
- Analizar la parametrización del crossover.
- Analizar la probabilidad de mutación.

Diseño del experimento:

El análisis de parámetros se realizó sobre el dataset con mayor número de ciudades: cities_100_131415. Se eligió ese archivo porque representa el escenario más exigente entre los datos disponibles. Para mantener un costo computacional razonable, cada configuración se evaluó con 40 generaciones y luego se aplicó 2-opt final. Las variables analizadas fueron tamaño de población, tipo de crossover, probabilidad de crossover y probabilidad de mutación.

Tabla 9. Diseño del análisis experimental de parámetros.

Parámetro analizado	Valores probados	Razón de análisis
Tamaño de población	60, 100, 140	Permite observar si más individuos mejoran exploración y calidad.
Tipo de crossover	OX y PMX	Ambos son válidos para permutaciones; se compara su desempeño experimental.
Probabilidad de crossover	0.80 y 0.90	Mide cuánto influye recombinar padres frente a

		copiar un padre.
Probabilidad de mutación	0.05 y 0.20	Evalúa el balance entre exploración adicional y perturbación excesiva.
Generaciones por configuración	40	Presupuesto uniforme para comparar parámetros bajo la misma restricción.

Tabla 10. Diez mejores configuraciones del experimento de parámetros.

Población	Crossover	Pc	Pm	Dist. AG+2opt	Error vs NN	Únicos finales (%)	Hamming final (%)
100	PMX	0.90	0.20	1,455.27	-8.53	24.0	18.2
60	OX	0.80	0.20	1,455.27	-8.53	21.7	12.0
140	OX	0.90	0.20	1,455.92	-8.49	25.0	13.8
60	PMX	0.80	0.20	1,460.41	-8.21	26.7	19.9
60	OX	0.80	0.05	1,461.26	-8.15	3.3	1.4
140	OX	0.90	0.05	1,462.68	-8.06	5.0	0.6
140	PMX	0.80	0.20	1,465.76	-7.87	21.4	4.9
100	PMX	0.90	0.05	1,465.76	-7.87	7.0	19.1
100	OX	0.90	0.20	1,467.45	-7.76	21.0	11.7
140	OX	0.80	0.20	1,468.00	-7.73	21.4	8.6

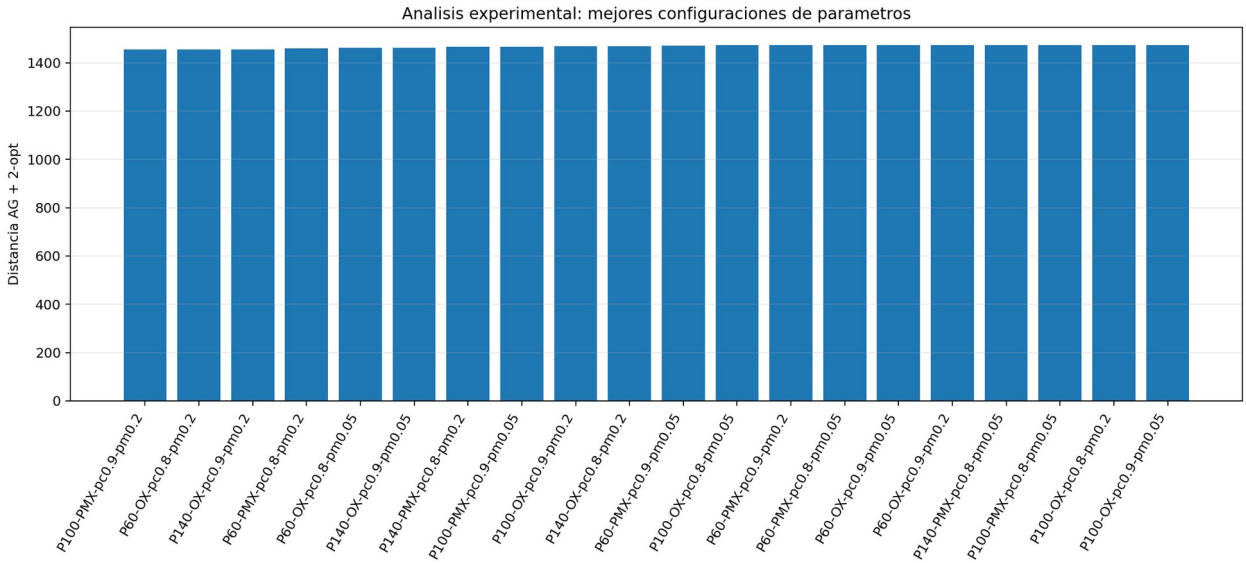


Figura 9. Mejores configuraciones del análisis experimental de parámetros.

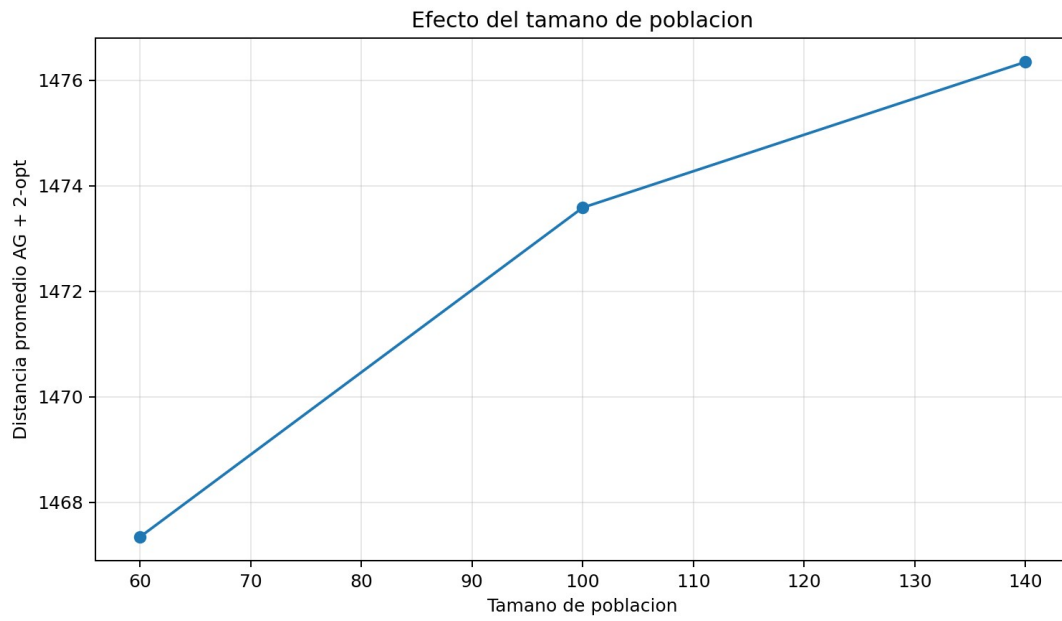


Figura 10. Efecto del tamaño de población sobre la distancia promedio.

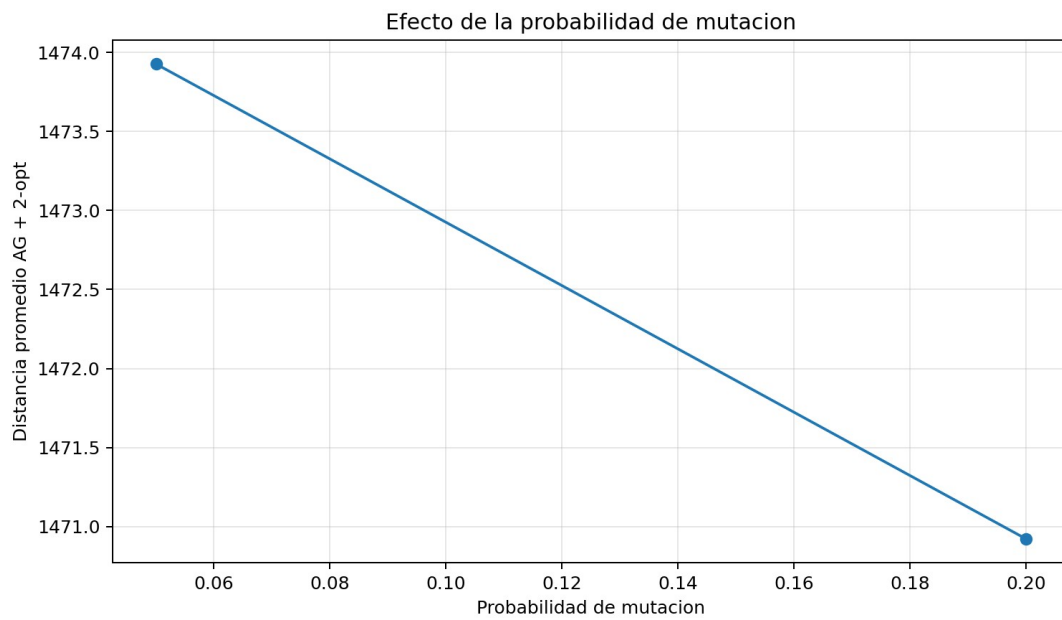


Figura 11. Efecto de la probabilidad de mutación sobre la distancia promedio.

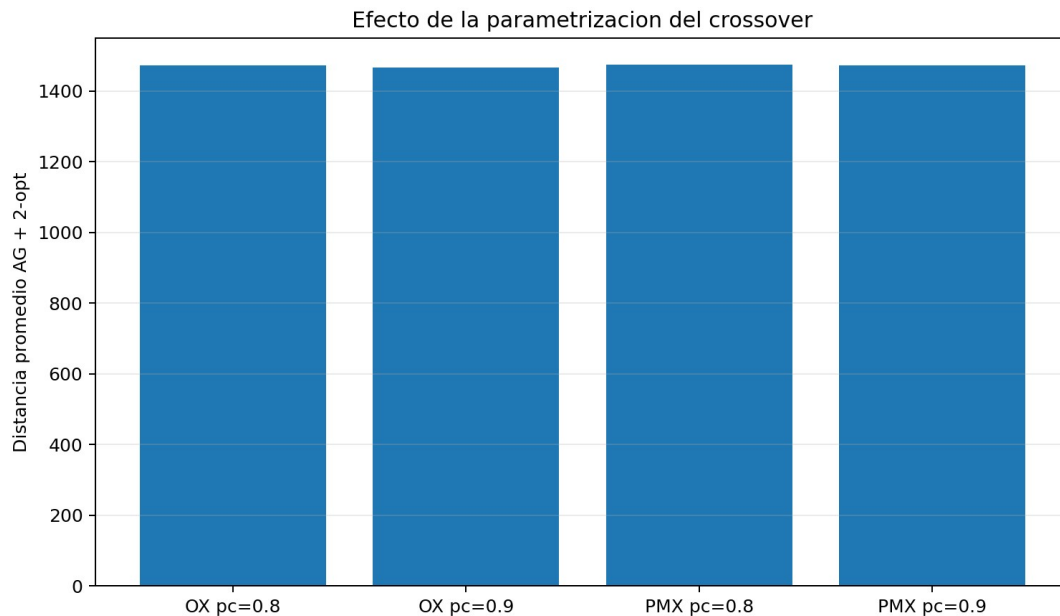


Figura 12. Efecto del tipo y probabilidad de crossover sobre la distancia promedio.

Tabla 11. Resumen por tamaño de población.

Población	Distancia promedio	Únicos prom. (%)	Hamming prom. (%)
60.00	1,467.34	14.8	8.6
100.00	1,473.58	17.2	12.3
140.00	1,476.35	15.7	8.3

Tabla 12. Resumen por probabilidad de mutación.

Prob. mutación	Distancia promedio	Únicos prom. (%)	Hamming prom. (%)
0.05	1,473.93	7.6	6.0
0.20	1,470.92	24.2	13.5

Tabla 13. Resumen por parametrización del crossover.

Crossover	Pc	Distancia promedio	Únicos prom. (%)
OX	0.80	1,473.38	17.0
OX	0.90	1,467.41	15.6
PMX	0.80	1,475.60	16.6
PMX	0.90	1,473.31	14.5

Análisis del tamaño de población:

Una población mayor tiende a incrementar la exploración porque mantiene más rutas candidatas por generación. Sin embargo, también aumenta el costo computacional. En el experimento, el tamaño poblacional no se interpreta de forma aislada, sino junto con mutación y crossover. Esto es importante para la defensa: una población de 140 puede explorar más, pero si la mutación es baja o el crossover no introduce suficiente variación, la mejora no será necesariamente proporcional.

Análisis del crossover:

OX y PMX son operadores válidos para permutaciones. OX fue elegido en la ejecución principal porque conserva el orden relativo de visita, que es una característica central en el TSP. El experimento incluyó PMX para demostrar que existen alternativas válidas y para comparar su efecto bajo el mismo presupuesto de generaciones. La probabilidad de crossover alta favorece la recombinación de rutas, pero si se usa sin mutación suficiente puede acelerar la homogeneización de la población.

Análisis de la mutación:

La mutación por inversión introduce exploración sin romper la validez de la ruta. Una probabilidad demasiado baja puede causar convergencia prematura porque la población deja de explorar. Una probabilidad demasiado alta puede dificultar la explotación de buenas rutas. En este experimento, 0.20 mostró configuraciones competitivas porque permitió recuperar diversidad en un problema donde el orden de visita es muy sensible.

Sustentación del literal f): el análisis no se limitó a mencionar parámetros; se probaron configuraciones, se reportaron tablas y se graficó el efecto de población, crossover y mutación. Esto permite defender por qué la configuración base es razonable y cómo podrían ajustarse los parámetros si se detecta estancamiento.

Conclusiones técnicas

1. La representación por permutación es la más adecuada para el TSP porque cada ruta debe contener todas las ciudades exactamente una vez. Esta representación evita modelar el problema como una cadena binaria, que sería menos natural para rutas ordenadas.
2. La validez de las rutas se garantizó en todas las etapas: inicialización mediante permutaciones, crossover OX mediante preservación y completado sin duplicados, y mutación por inversión mediante reordenamiento sin pérdida de genes.
3. El AG elitista fue una elección pertinente porque protege las mejores soluciones encontradas. Esto es especialmente útil en el TSP, donde una buena ruta puede perderse por operaciones aleatorias si no existe elitismo.
4. OX no compite con el AG elitista; OX es el operador de cruce dentro del AG. La estructura principal es el AG generacional elitista, mientras que OX permite recombinar rutas sin violar la restricción de permutación.
5. El fitness definido como inverso de la distancia permitió transformar un problema de minimización en una medición de maximización, lo que facilita la interpretación evolutiva del algoritmo.
6. Las gráficas de fitness y distancia evidencian mejora inicial rápida seguida de estabilización, comportamiento típico de algoritmos genéticos en problemas combinatorios.
7. La diversidad poblacional disminuyó conforme la población convergió hacia mejores rutas. Esta reducción es normal, pero debe vigilarse porque puede producir convergencia prematura en algunos datasets.
8. La comparación contra Nearest Neighbor demuestra que el AG elitista + 2-opt obtuvo mejores distancias en todos los datasets respecto a la heurística simple.
9. El uso de 2-opt como refinamiento final fortaleció la calidad de las rutas al corregir segmentos locales ineficientes; por ello, el enfoque puede describirse como AG elitista con mejora local final.
10. El análisis de parámetros mostró que la calidad final depende del equilibrio entre exploración y explotación. Población, crossover y mutación deben seleccionarse conjuntamente, no de forma aislada.
11. El proyecto cumple los literales a) a f) porque incluye representación válida, operadores justificados, gráficas de fitness, métricas de diversidad, comparación con heurística y experimentación de parámetros con tablas y gráficas.
12. Una posible mejora futura sería ejecutar múltiples semillas por configuración para estimar variabilidad estadística, así como comparar con otras heurísticas como inserción más cercana, simulated annealing o colonia de hormigas.

Resumen de participación del Grupo 5

Tabla 14. Participación equilibrada de los integrantes del Grupo 5.

Integrante	Responsabilidad asignada	Actividades realizadas
Nancy Altamirano	Preparación de datos, representación y validación	Organizó los CSV, verificó columnas city, x, y, controló cantidad de ciudades por dataset, apoyó el literal a), validó rutas sin ciudades repetidas ni faltantes y revisó la consistencia de las tablas de validación.
Gustavo Berru	Diseño e implementación del algoritmo genético	Implementó la estructura del AG generacional elitista, selección por torneo, elitismo, crossover OX, mutación por inversión, validaciones de permutación, registro de histórico por generación y exportación de

		rutas finales.
Raquel Pacheco	Evaluación de calidad y análisis experimental	Implementó y revisó la comparación contra Nearest Neighbor, cálculo de fitness final, error relativo, análisis del dataset más grande, experimentos de población, crossover y mutación, y consolidación de resultados cuantitativos.
Kevin Viteri	Análisis gráfico, diversidad y documentación final	Generó las gráficas de fitness, convergencia, diversidad, comparación y parámetros; analizó estancamiento y convergencia prematura; integró la documentación profesional y revisó ortografía, estructura y sustentación para exposición.

La distribución se planteó de manera balanceada para que cada integrante participe en una parte técnica del proyecto: datos y representación, algoritmo, evaluación experimental, y análisis/documentación. Esto permite defender el trabajo como un desarrollo grupal integral y no como una división basada en tareas menores.

Referencias de apoyo

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791-812.

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.

Reinelt, G. (1994). *The traveling salesman: Computational solutions for TSP applications*. Springer.

Whitley, D. (1994). A genetic algorithm tutorial. *Statistics and Computing*, 4, 65-85.